Deep Learning

Activation Function

Equipping You with Research Depth and Industry Skills

By:

Dr. Zohair Ahmed

Why Need an Activation Function?

- **Why Do Neural Networks Need an Activation Function?**
- Imagine you're building a smart machine that can learn patterns like recognizing faces, detecting spam, predicting prices, or translating languages.
- A neural network learns these patterns by connecting many neurons (like brain cells).
  Each neuron does two basic things:
  - **Takes input data**
  - **Applies a mathematical transformation**
- Without an activation function, this transformation is **only linear** and that's a big problem.

Without Activation Functions = Only Straight Lines

- If your neural network has **no activation functions**, then each layer just does:
  - output = W * input + b
- That's a *linear* transformation basically drawing a straight line.
- Even if you add 100 layers, you still get *just another straight line*.
  - Multiple linear layers combined ≡ one linear layer
    Mathematically: Composition of linear functions = still linear
    No ability to learn curves, shapes, or complex patterns
- So your entire neural network becomes nothing more than: **a fancy linear regression model.**

Why Need an Activation Function?

Most Real-World Problems Are Non-linear

- Life isn’t straight lines.
  - Classifying cats vs dogs → curved boundary
  - Predicting disease from symptoms → highly non-linear
  - Translating Urdu to English → extremely complex
  - Recognizing handwritten digits → weird shapes, variations
- To learn such patterns, we need **non-linearity**.
- And that’s exactly what activation functions provide.

Why Need an Activation Function?

Activation Functions Add "Bend" to the Network

- Activation functions like **ReLU, Sigmoid, Tanh** add a non-linear step:
  - output = activation(W * input + b)
- This allows the network to learn:
  - Curves
  - Corners
  - Complex boundaries
  - Multidimensional shapes
  - Abstract patterns
- **Simple Analogy**
- Think of linear layers as straight sticks. Activation functions allow you to **bend**, **twist**, and **shape** the sticks.
- Only then can you build complex structures.

Why Need an Activation Function?

Without activation function:

- Line → Line → Line → still a line
- Can only learn:
  - Straight boundaries
  - Simple patterns

With activation function:

- Line → **curve** → **complex curve** → **shape**

  Can learn:
  - Circles
  - Spirals
  - Faces
  - Speech patterns
  - Meaning of words (NLP)

Binary Step Function

- The **binary step function** is one of the earliest activation functions, used mainly in **perceptrons**.
- It works like an **ON/OFF switch**:
- If the input is **less than 0** → output = **0** (OFF)
- If the input is **greater than or equal to 0** → output = **1** (ON)

$$f(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

- It simply checks:
- **Is the input big enough?**

  If yes → activate

  If no → stay off

Why Binary Step Is NOT Used in Deep Learning

- Although it is simple, it has **major limitations**.

1. ❌ No multi-value output

- Binary step can output ONLY: **0** or **1**
- This means it **cannot** handle:
  - multi-class classification
  - regression
  - probability estimation
- It's too limited for real-world problems.

2. ❌ No gradient for backpropagation

- Deep learning uses **gradient descent** to learn.
  But the binary step function has:
- **zero gradient** everywhere
- a **jump (discontinuity)** at 0
- This makes learning **impossible**.
- Because backpropagation needs a slope to update weights.
- But binary step's slope = **0**:
- No slope → no learning
- No learning → no deep learning
- This is the biggest reason it is not used today.

3. ❌ Not sensitive to small changes

- If:
- input = 0.0001 → output = 1
- input = 1000 → output = 1
- Both produce exactly the same output.
- So the function loses all information about *how strong* the input is.

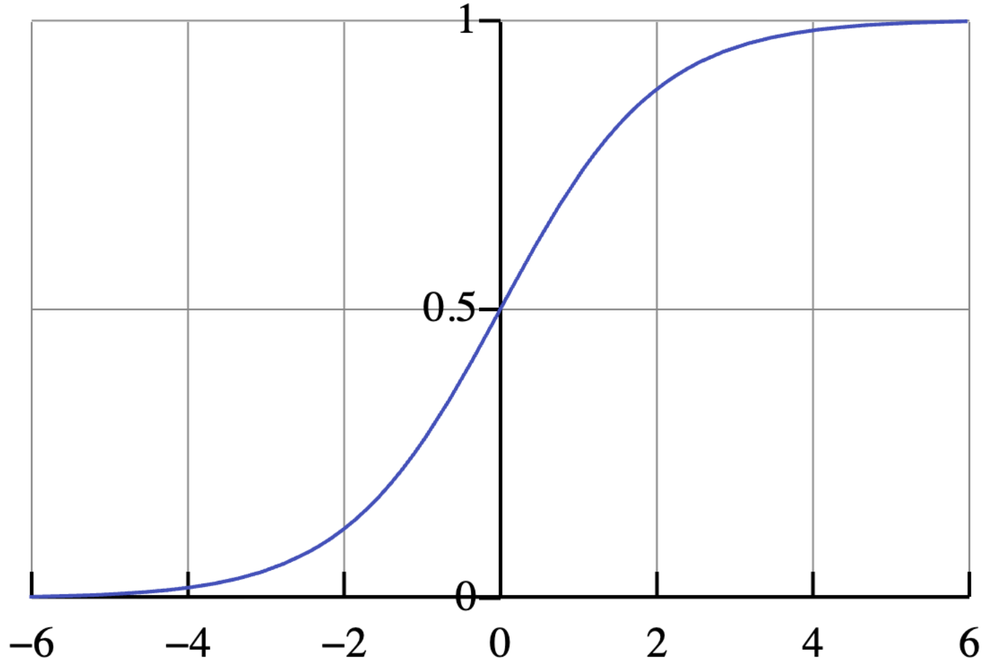
Linear Activation Function (Identity Function)

No matter how many layers you add, the entire network is STILL linear

- This is the biggest issue, If every layer is linear:
- $f(x) = W_3(W_2(W_1x + b_1) + b_2) + b_3$
- You can combine all weights into one:
- $f(x) = Wx + b$
  All layers collapse into **one single linear transformation**.
- **So a deep neural network with linear activations is basically:**
- A **single-layer linear regression model** even if you built 100 layers!
- Depth becomes meaningless
  - ❌ Cannot learn complex patterns
  - ❌ Cannot model curved boundaries
  - ❌ Cannot solve real-world tasks

1. Sigmoid / Logistic Activation Function

- The **sigmoid activation function** takes any real number and pushes it into a range between: 0 and 1
- The mathematical form is: $\sigma(x) = \frac{1}{1+e^{-x}}$
- If **x is large positive**, output → **1**
- If **x is large negative**, output → **0**
- If **x is around 0**, output → **0.5**
- This is why the sigmoid curve looks like a smooth **S-shaped** graph.



Why is Sigmoid Used?

1. Great for probability predictions

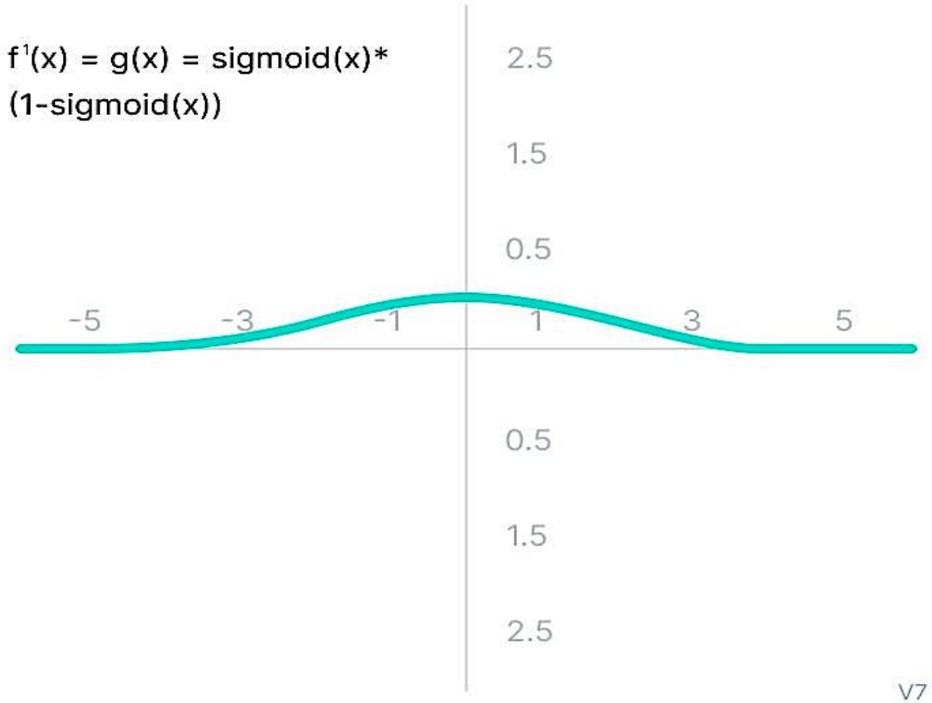
- Probabilities must be between **0 and 1**. Sigmoid naturally outputs values in this range, so it is widely used in:
  - Binary classification
  - Logistic regression
  - Last layer of binary classifiers
- Example: “Is the email spam?”
- → Output 0.94 → 94% chance of spam

Limitations of Sigmoid: Vanishing Gradient

- Although sigmoid was popular in early neural networks, it has several major drawbacks.

❌ **1. Vanishing Gradient Problem**

- This is the biggest issue.
- Look at how the sigmoid behaves:
  - For inputs **> +3**, the curve becomes flat
  - For inputs **< −3**, the curve becomes flat
- Here's what the image tells you:
  - Around x = 0
  - The gradient is **highest (peak of the hill).**
  - This means sigmoid **learns best near 0.**
- Around x > +3 or x < –3
  - The gradient is almost zero (flat).
  - This means sigmoid learns almost nothing in these regions.



Vanishing Gradient Problem

- Input x= [1.0, 2.0, -1.0]
- These are normal small numbers nothing close to ±3 yet.

Step 1: Apply the neuron calculation

- A neuron computes: $z = Wx + b$
- Let's choose simple weight and bias:
- Weight **W = 3** , Bias **b = 0**
- Now calculate **z** for each x.

Step 2: Multiply and see the results

- **For x = 1.0** $\rightarrow z = 3 \cdot 1 = 3$
- **For x = 2.0** $\rightarrow z = 3 \cdot 2 = 6$
- **For x = -1.0** $\rightarrow z = 3 \cdot (-1) = -3$

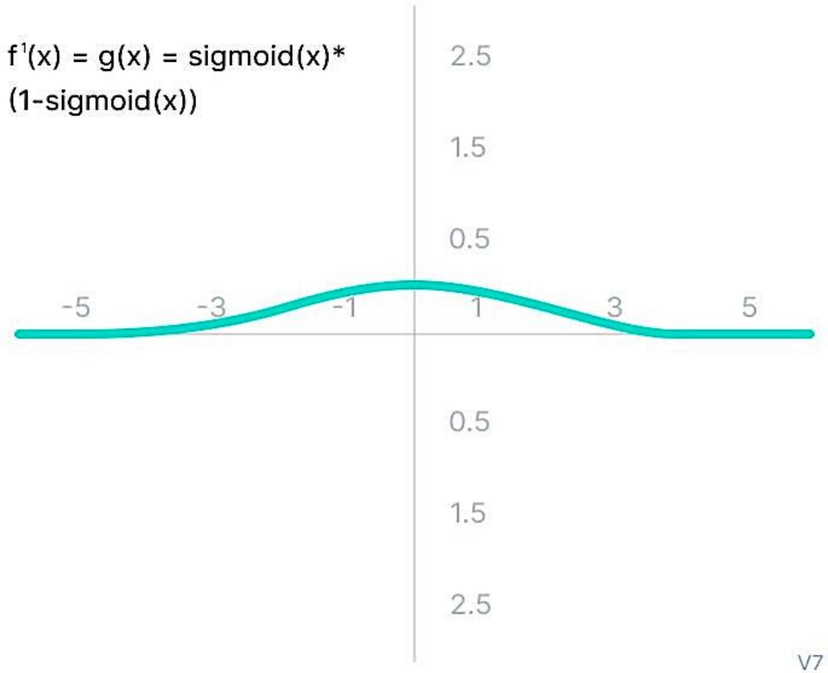
Look what happened!

- The input values were: **1, 2, -1**
- But inside the neuron, they became: **3, 6, -3**

- Now we have **+3 and -3 exactly.**
- This is how the internal value (z) reaches the dangerous sigmoid/tanh flat regions.

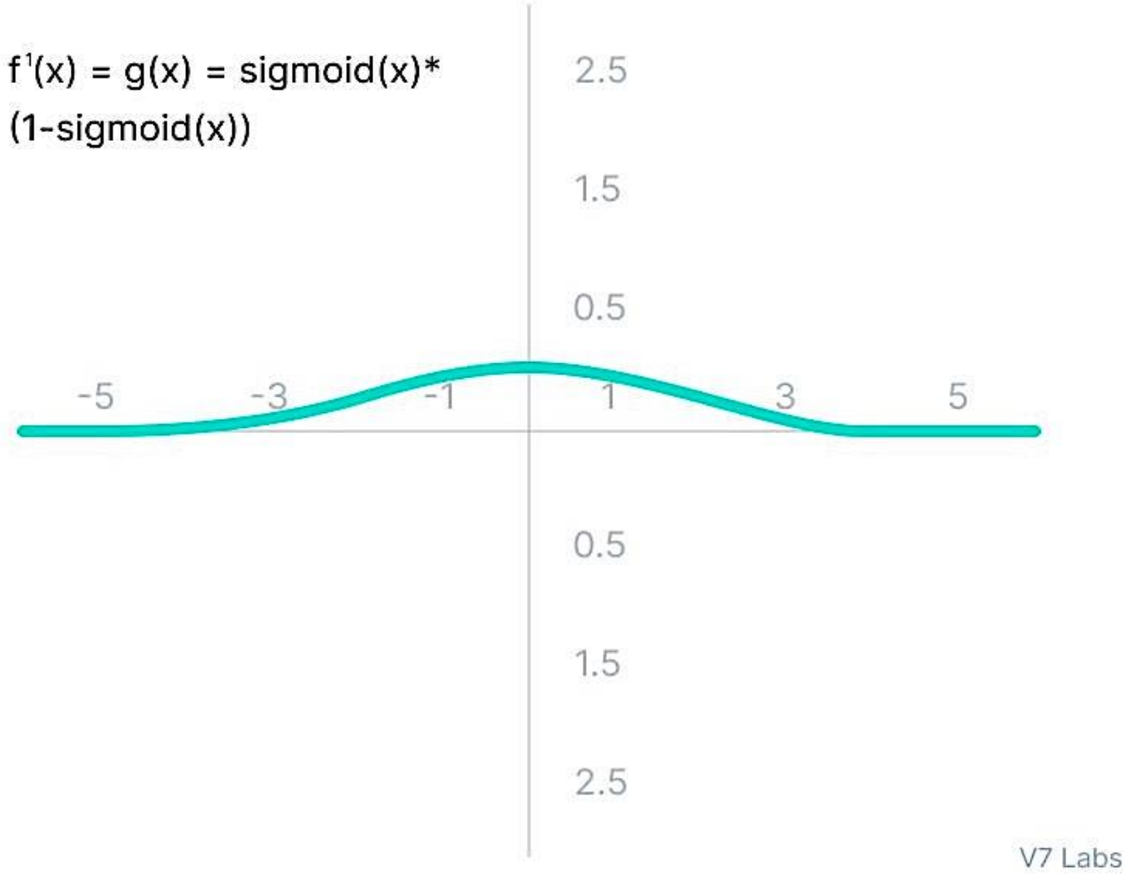
Step 3: Pass z into sigmoid/tanh

- **For z = 3**
- Sigmoid output ≈ 0.95
  Derivative ≈ 0.047 → **tiny**
- **For z = 6**
- Sigmoid output ≈ 0.997
  Derivative ≈ 0.002 → **almost zero**
- **For z = -3**
- Sigmoid output ≈ 0.047
  Derivative ≈ 0.044 → **tiny**



Why this causes Vanishing Gradient

- Look at the derivatives :
  - 0.047
  - 0.002
  - 0.044
- These are VERY small numbers.
- During backpropagation, small derivatives multiply together:
- $0.04 \times 0.03 \times 0.02 \times 0.01 \rightarrow 0.00000024$
- This **kills learning** in deep networks.



Limitations of Sigmoid

- The derivative (slope) ≈ 0
- The graph is lying almost horizontally
- Weight updates become tiny
- Learning becomes very slow or stops completely
- When x = 4, 5, 6 → sigmoid output is almost 1, gradient ≈ 0
- When x = –4, –5, –6 → sigmoid output is almost 0, gradient ≈ 0
- This “flatness” on both sides is what causes the vanishing gradient problem.

During backpropagation:

- Weight update = **gradient** × **learning rate**
- If gradient = **0** → weight update = **0**
- **The neuron stops learning.**
- That’s why sigmoid is not used in deep hidden layers anymore.

Limitations of Sigmoid

❌ 2. Output is not zero-centered

- Sigmoid outputs are always **positive**:
- $0 < \sigma(x) < 1$
- Neuron outputs always have the same sign
- Gradient updates become biased
- Causes **zig-zagging** during optimization
- Training becomes slow and unstable.

Think of Gradient Like a Slope

- Imagine you are standing on a hill.
  - If the ground is steep → slope (gradient) is **large**
  - If the ground is flat → slope (gradient) is **zero**
- **In neural networks:**
  - **Large gradient → big learning step**
  - **Small gradient → tiny learning step**
  - **Zero gradient → no learning**
- Zero-centered activations (like **tanh**) fix this problem.

❌ 3. Sigmoid is computationally expensive

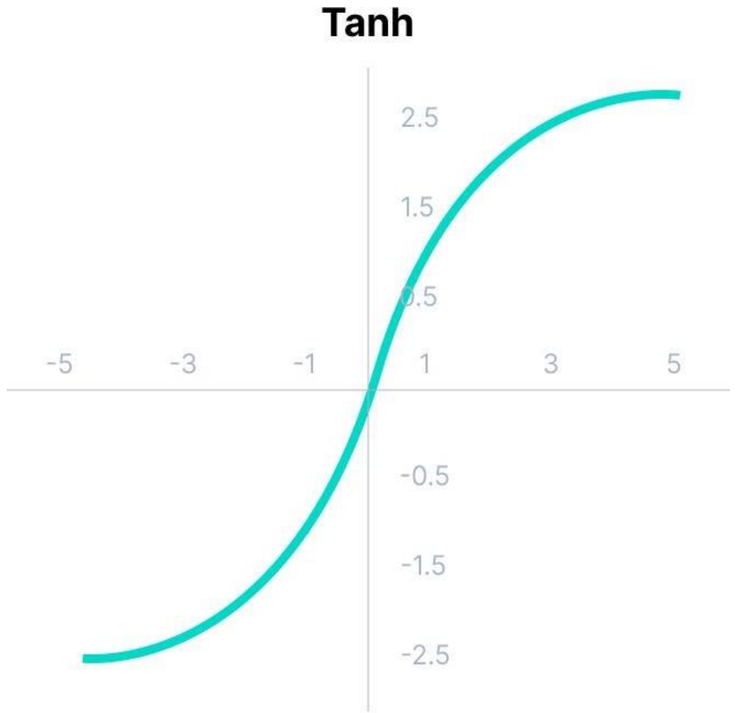
- It uses the exponential function: e^{-x}
- This is slower to compute compared to simple functions like ReLU.
- Today, efficiency matters especially on large networks.

What is the Tanh Activation Function?

- Tanh is very similar to sigmoid, but:
- **Sigmoid outputs:** $0\ to\ 1$
  Tanh outputs: $-1\ to\ 1$
- If input is **positive big**, output → **1**
- If input is **negative big**, output → **–1**
- If input is **around 0**, output → **0**
- That's why tanh looks like a stretched sigmoid.
- **Why Tanh is Better than Sigmoid**

1. Tanh is Zero-Centered

- Because tanh outputs can be:
- strongly negative → –1
- neutral → 0
- strongly positive → +1
- This makes the **mean output of neurons close to 0**, which helps the next layer learn better.
- It avoids the "always positive output" problem of sigmoid.
  - This avoids biased gradient directions.
  - It reduces zig-zagging in gradient descent.
  - Training becomes smoother and faster.



$$Tanh$$

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

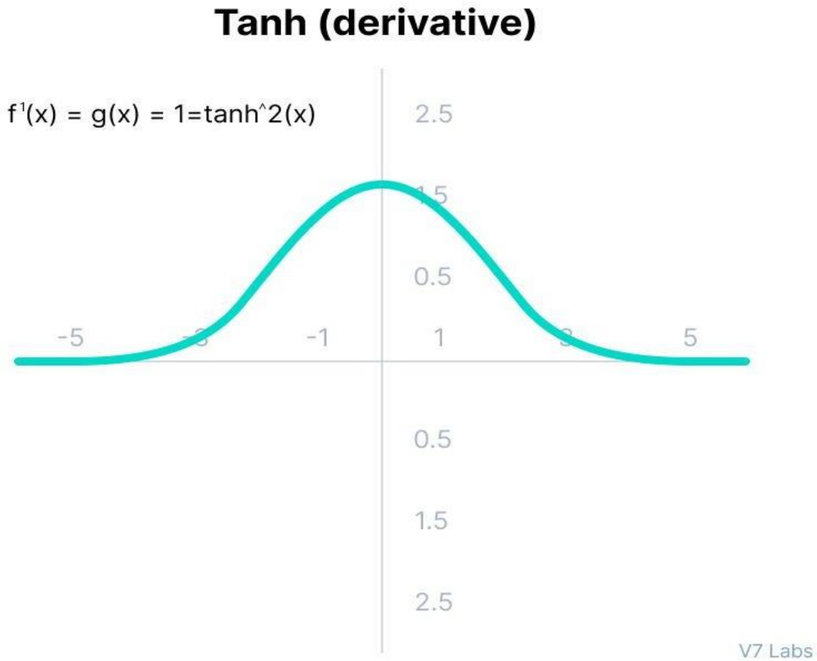
What is the Tanh Activation Function?

2. Helps in Centering the Data

- Neural networks learn better when data is centered around zero.
- Since tanh outputs are between –1 and +1:
- The data fed to the next layer is balanced
- Optimization becomes easier
- Convergence (learning) is faster
- This is why tanh is often used in **hidden layers**.

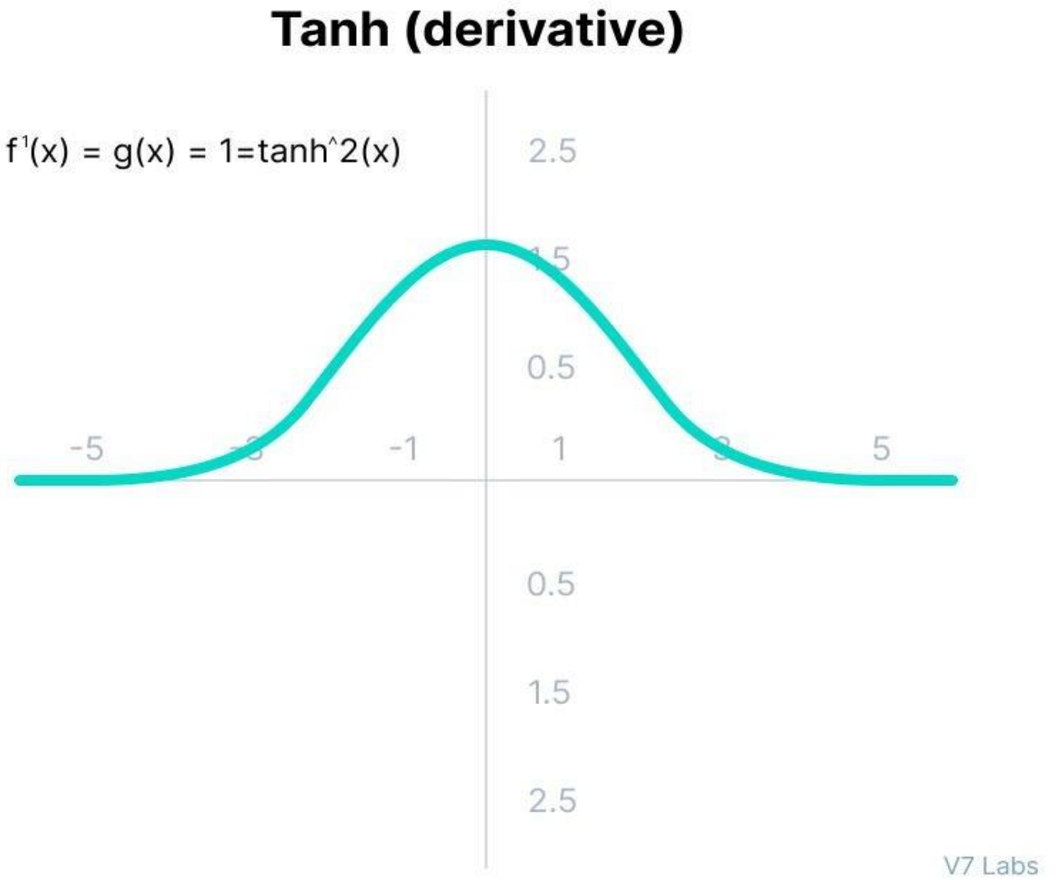
Gradient of Tanh Activation Function

- The derivative of tanh is: $1 - \tanh^2(x)$
- This is represented a **bell-shaped curve**.
- Gradient is biggest around **0**
- Gradient becomes very small for **x > +3** or **x < –3**



Vanishing Gradient Problem (Tanh also suffers!)

- Just like sigmoid:
- When inputs are very positive or very negative
- The tanh curve becomes flat
- Gradient → **almost zero**
- Weight updates → **tiny**
- Learning → **almost stops**
- So yes, **tanh still suffers from vanishing gradients**, especially in deeper networks.



Tanh Is Still Preferred Over Sigmoid Why?

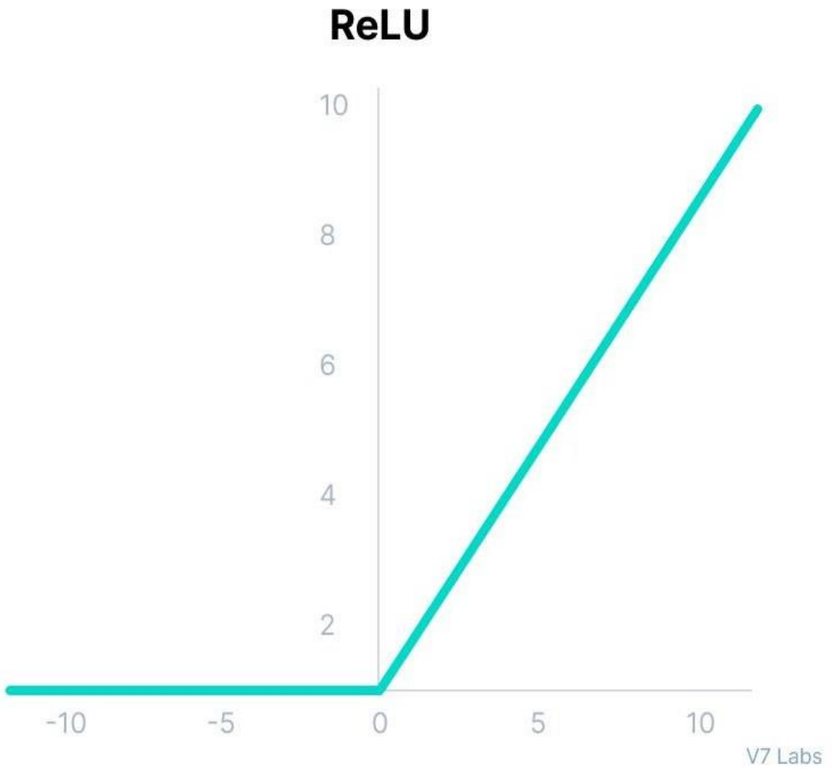
- Even though **both** have vanishing gradient problem…
- **Tanh is zero-centered**
  - Outputs can be negative or positive.
- **Gradients can move in both directions**
  - This makes optimization smoother.
- **Less zig-zagging**
  - Unlike sigmoid (which only outputs positive values), tanh does not push gradients in only one direction.
- **Faster learning in practice**
  - Hidden layers with tanh typically train faster and more stable than sigmoid layers.
- So in practice:
- **Tanh is always preferred over Sigmoid for hidden layers.**
- **Sigmoid output:** 0.1, 0.3, 0.5, 0.7, 0.9
  - (Always positive)
- **Tanh output:** -0.8, -0.2, 0, 0.3, 0.9
  - (Positive AND negative → centered at 0)
- Tanh gives the next layer more balanced inputs, making learning easier.

Tanh Is Still Preferred Over Sigmoid Why?

- Even though **both** have vanishing gradient problem…
- **Tanh is zero-centered**
  - Outputs can be negative or positive.
- **Gradients can move in both directions**
  - This makes optimization smoother.
- **Less zig-zagging**
  - Unlike sigmoid (which only outputs positive values), tanh does not push gradients in only one direction.
- **Faster learning in practice**
  - Hidden layers with tanh typically train faster and more stable than sigmoid layers.
- So in practice:
- **Tanh is always preferred over Sigmoid for hidden layers.**
- **Sigmoid output:** 0.1, 0.3, 0.5, 0.7, 0.9
  - (Always positive)
- **Tanh output:** -0.8, -0.2, 0, 0.3, 0.9
  - (Positive AND negative → centered at 0)
- Tanh gives the next layer more balanced inputs, making learning easier.

What is ReLU?

- ReLU means **Rectified Linear Unit**.
- $f(x) = \max(0, x)$
- This means:
- If **x > 0**, output = x
- If **x < 0**, output = 0
- A flat line for negative values
- A straight rising line for positive values



$$ReLU$$

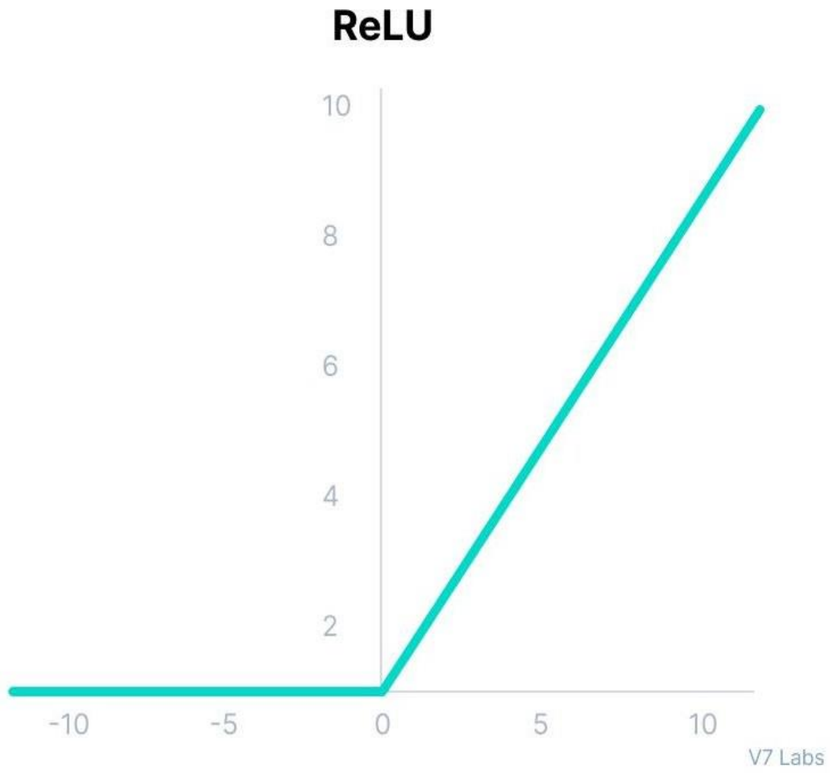
$$f(x) = max\,(0, x)$$

Very fast (super cheap to compute)

- Unlike sigmoid and tanh, ReLU does **not** use exponentials.

  It only uses a simple comparison:

- Is x > 0?
- If yes → output x
- If no → output 0
- This makes ReLU **much faster**.



Avoids vanishing gradient (mostly)

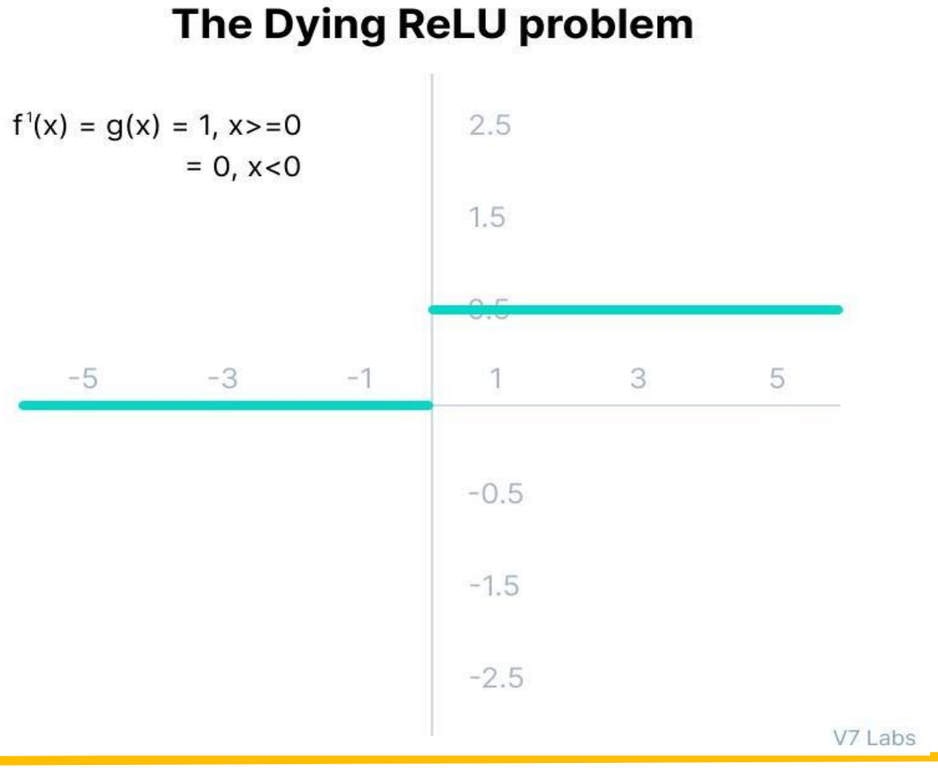
- For x > 0:
- $f'(x) = 1$
- This means:
- The gradient does not shrink
- Gradients flow nicely through deep networks
- Learning is faster
- Convergence to minimum is quicker
- This is why ReLU is used in almost all modern neural networks (CNNs, transformers, etc.).
- **Only some neurons activate → sparsity**
- If x < 0, output is zero.
- This means many neurons do nothing (inactive).

  This **reduces computation** and makes the network more efficient.

ReLU has a Big Problem “Dying ReLU”

- The derivative graph of ReLU.
- Derivative of ReLU is: $f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$
- The part where x < 0 has gradient = **0**.
- **What does gradient = 0 mean?**
- You already know:
- **Large gradient → big learning**
- **Small gradient → tiny learning**
- **Zero gradient → no learning**

- So if ReLU outputs **0** for negative x, gradient becomes **0** during backprop.
- This means the neuron **stops learning**.



Why is this a problem?

1. The neuron becomes “dead”

- If a neuron’s output becomes 0 for many inputs, it will always stay 0 forever, because:
- gradient = 0
  - weights stop updating
  - neuron never activates again
  - That neuron becomes **dead**.
  - It stops participating in learning.

2. Network capacity decreases

- If many neurons die, the network loses representation power.
- The model may:
  - underfit
  - learn slowly
  - fail to capture patterns
  - perform poorly

3. Negative values instantly become zero

- This means ReLU throws away ALL negative information.
- Example:
  - input = -1 → output = 0
  - input = -5 → output = 0
  - input = -0.0001 → output = 0
- All treated the same.
- This reduces flexibility.

Why does ReLU still dominate?

- Even with the dying ReLU problem, it is still preferred because:
  - Very fast
  - Easy to compute
  - Helps gradients flow
  - Works great in deep networks
- In practice, most neurons do NOT die if learning rate is good.

What is Leaky ReLU?

- Leaky ReLU is a small improvement over ReLU.
- ReLU does this: $\text{ReLU}(x) = \max(0, x)$
- But **Leaky ReLU does this**:
- $\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x < 0 \end{cases}$

  For positive values → acts like normal ReLU
- **For negative values → instead of zero, it gives a *small negative slope***
- Example:
- ReLU(-5) = 0
- LeakyReLU(-5) = -0.05
- This tiny leak (0.01x) allows the neuron to **still learn** even when input is negative.

Why Leaky ReLU was created?

- It was created to solve the **Dying ReLU Problem**.
- **ReLU negative side:**
- $f'(x) = 0$(no learning)
- So neurons “die” because they never update again.

Limitations of Leaky ReLU

- Even though Leaky ReLU is better, it still has problems.

❌ **1. Predictions for negative inputs may be unstable**

- Because the slope (0.01) is small and arbitrary.
- Should it be 0.01?, 0.02?, 0.1?
- There is no perfect choice.

❌ **2. Gradient is still small → learning is slow on negative side**

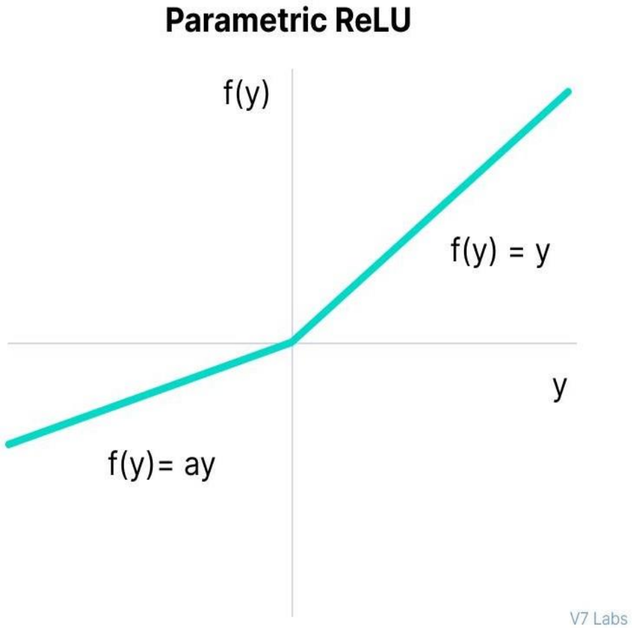
- For negative values:
- gradient $= 0.01$
- This is **very small**, so learning happens slowly on that side.
- Much slower than the positive side where gradient = 1.
- **This is why PReLU (Parametric ReLU) was created**
- In PReLU, that “0.01” is learned automatically by the network.

What is Parametric ReLU (PReLU)?

- Parametric ReLU is an improved version of ReLU and Leaky ReLU.
- $f(x) = \begin{cases} x, & x > 0 \\ ax, & x < 0 \end{cases}$
- Where:
- **x** = input
- **a** = a *learnable parameter* (a small slope on the negative side)
- **Important difference** from Leaky ReLU:
  In Leaky ReLU, the negative slope (0.01) is **fixed**.
  In PReLU, the negative slope **a is learned during training**, just like weights.

In the PReLU graph:

- For positive values: line goes up → slope = 1
- For negative values: line also goes up slightly → slope = **a** (not zero)
- This is why the left side is not flat like ReLU.



Why was PReLU created?

- **To solve the Dying ReLU Problem**
- ReLU outputs zero for all negative inputs → gradient = 0 → neuron “dies”.
- **To fix limitations of Leaky ReLU**
- Leaky ReLU uses a **fixed** slope like 0.01.
- But what if 0.01 is not the best choice for your dataset?
- **PReLU learns the best negative slope automatically**

- During backpropagation, the network learns the value of **a**:
  - Maybe the best slope is 0.001
  - Maybe 0.02
  - Maybe 0.2
  - Maybe 0.5
- The model figures it out by itself.

Limitations of PReLU

- Even though PReLU is powerful, it has some drawbacks:

❌ **1. Performance depends on the slope parameter “a”**

- If the learned slope becomes too large, the negative side may behave strangely.
- Different tasks may need different slopes.

❌ **2. More parameters to learn**

- Every neuron or channel may have its own “a” value → more memory, more computation.
- Not a big issue, but still something to consider.

❌ **3. Can overfit on small datasets**

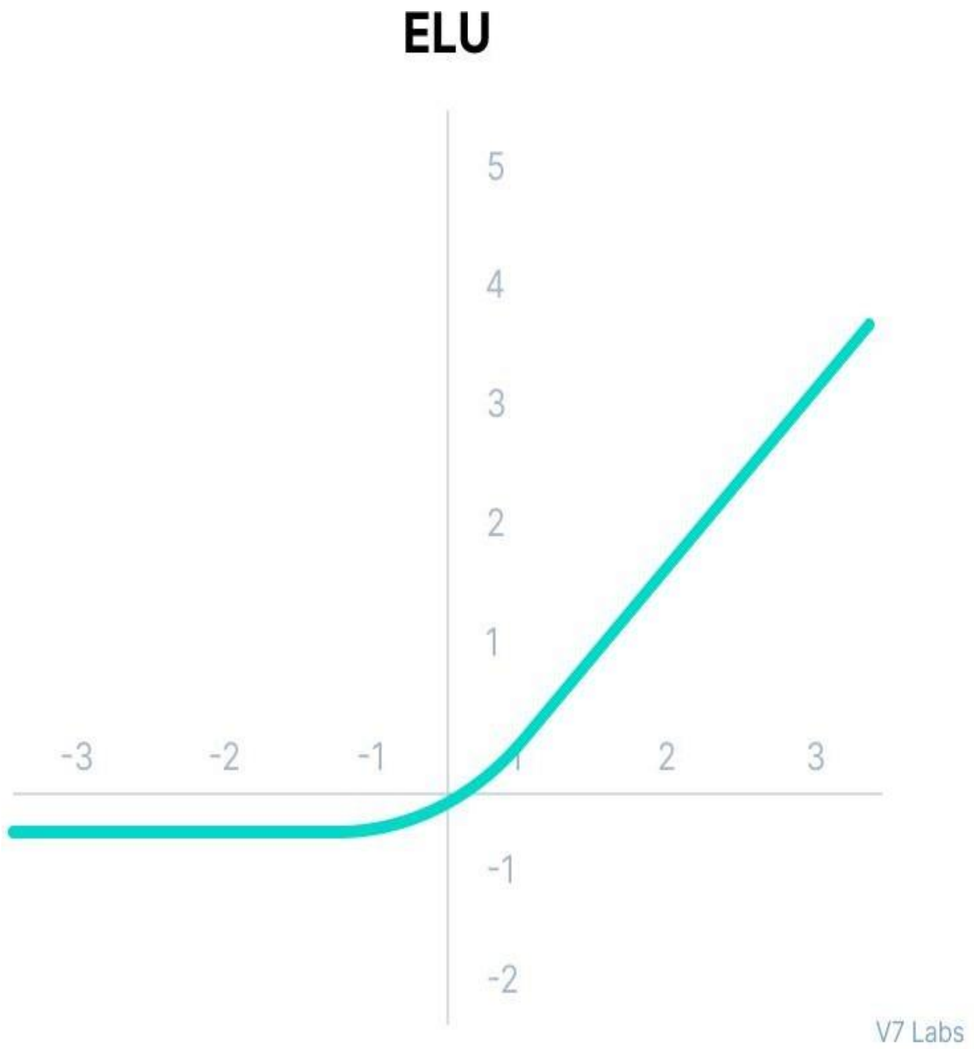
- Because the model is more flexible, it may fit noise on small datasets.

What Is the ELU Activation

- ELU stands for **Exponential Linear Unit**.
  It is another improvement over ReLU created to fix:
- Dying ReLU problem
- Zero-centered output issue
- Harsh cutoff for negative values
- **ELU formula (α = 1):**

- $$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

  This means:
- Positive side → same as ReLU
- Negative side → soft exponential curve (not straight line)
- Your graph shows exactly this:
  A smooth curve for negative x, then a straight rising line for positive x.



Intuition: What Does ELU Do?

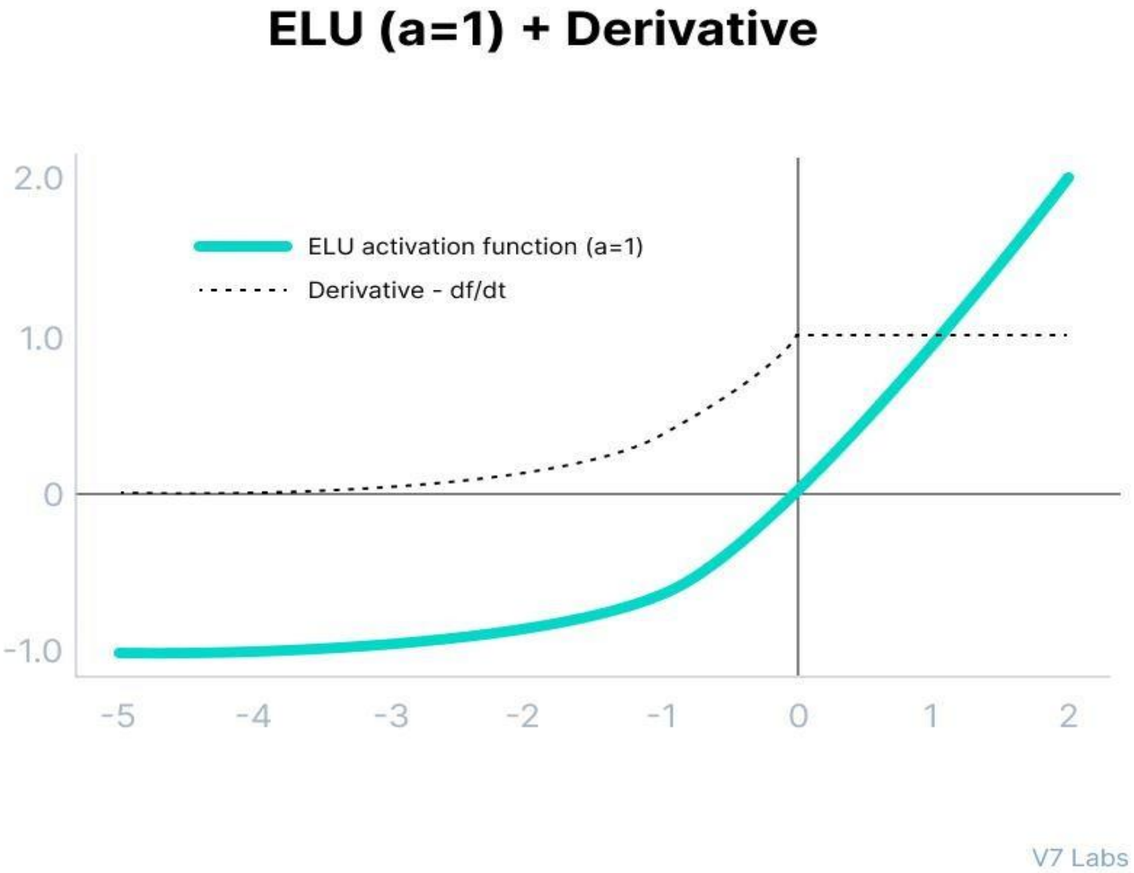
- **For positive inputs**
- ELU behaves just like ReLU:
- Output = x
- Gradient = 1
- No saturation
- Fast learning
- **For negative inputs**
- ELU smoothly curves down using exponential function:
- Never becomes flat
- Negative outputs slowly approach –α
- Gradient is non-zero
- Neuron never “dies”
- This is why ELU is preferred over ReLU in some networks.

Intuition: What Does ELU Do?

Understanding the Graph

- **ELU function (α = 1)**
- Right side: straight line (no change, same as ReLU)
- Left side: smooth decreasing curve until –1
- **Dotted line → derivative of ELU**
- Derivative = 1 when x > 0
- Derivative slowly decreases on the negative side, but never becomes 0

- This “smoothness” helps ELU learn better than ReLU.



Compare Them Clearly

ReLU

- x > 0 → x
- x ≤ 0 → 0
- Negative side = ❌ dead

  Positive side = ✓ linear

Leaky ReLU

- x > 0 → x
- x ≤ 0 → 0.01x (small slope)
- Negative side = small straight line

PReLU (Parametric ReLU)

- x > 0 → x
- x ≤ 0 → a x (slope a is learned)
- Negative side = straight line *but slope is learned automatically*

ELU (Exponential Linear Unit)

- x > 0 → x
- x ≤ 0 → α(e^x - 1)

❗ **ELU uses EXponential curve on negative side**

- Not a straight line like Leaky ReLU or PReLU.

Mini Example ELU

Let α = 1

Input x	ELU(x)	f'(x)
2	2	1
0	0	1
–1	$e^{-1} - 1 = -0.63$	$e^{-1} = 0.367$
–3	$e^{-3} - 1 = -0.95$	$e^{-3} = 0.049$
–5	$e^{-5} - 1 = -0.993$	$e^{-5} = 0.006$

Notice:

- Outputs are negative but not flat
- Gradients never reach zero
- Neuron continues learning
- No dead neurons

Final Summary of ELU

- ELU = ReLU on positive side
- ELU = smooth exponential curve on negative side
- Zero-centered → better learning
- Avoids dead neurons
- Good alternative to ReLU

BUT

- ❌ Slower (exponential cost)
- ❌ Can cause exploding gradients
- ❌ Alpha is fixed

Limitations of ELU

❌ **1. Slower computation**

- ELU uses: e^x
- Exponentials are expensive compared to ReLU's max(0, x).

❌ **2. Alpha (α) is fixed, not learnable**

- Unlike Parametric ReLU (PReLU), ELU does not learn the slope.

❌ **3. Can cause exploding gradients**

- Because:
  - Positive side grows linearly (like ReLU)
  - Negative side increases exponentially
  - Together can cause very large gradients
  - Batch Normalization is usually added to prevent this.

Limitations of ELU

❌ **1. Slower computation**

- ELU uses: e^x
- Exponentials are expensive compared to ReLU's max(0, x).

❌ **2. Alpha (α) is fixed, not learnable**

- Unlike Parametric ReLU (PReLU), ELU does not learn the slope.

❌ **3. Can cause exploding gradients**

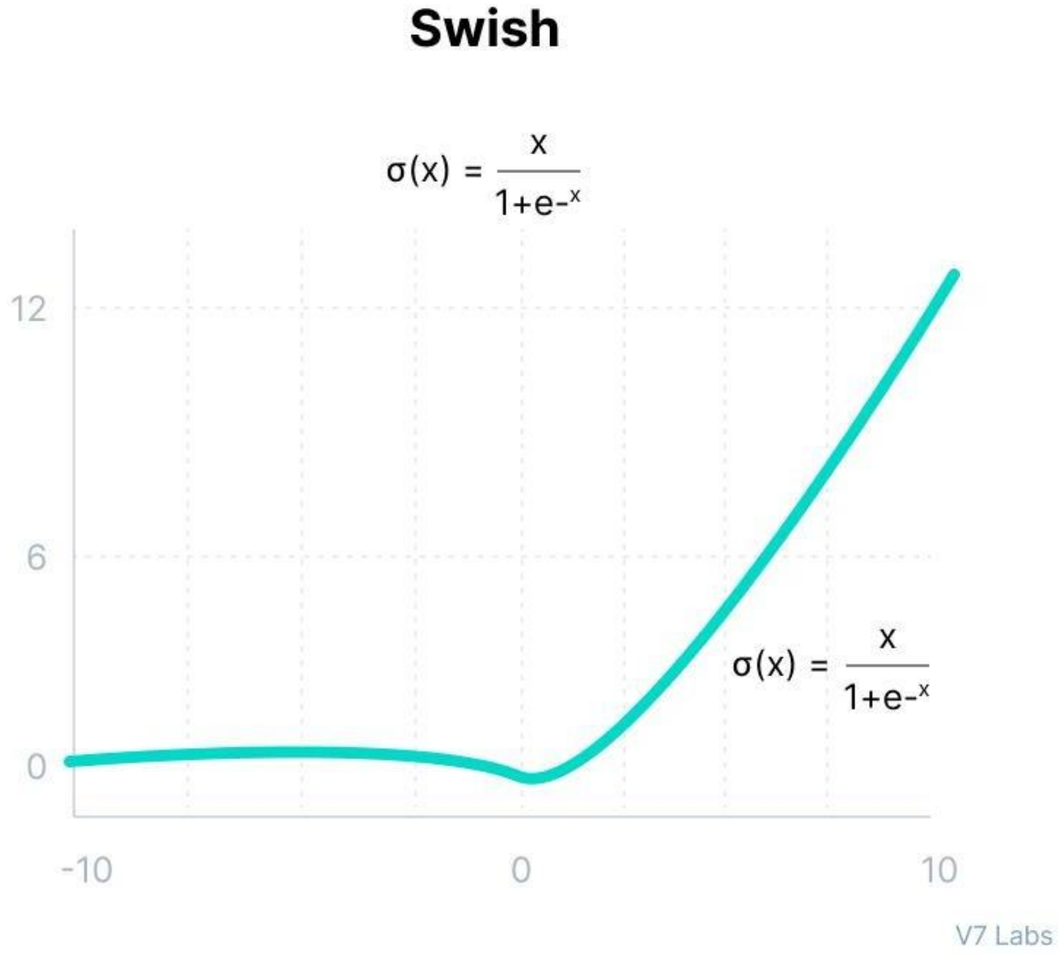
- Because:
  - Positive side grows linearly (like ReLU)
  - Negative side increases exponentially
  - Together can cause very large gradients
  - Batch Normalization is usually added to prevent this.

What is the Swish Activation Function?

- Swish is a **self-gated activation function** **networks.** created by **Google Brain researchers**.
- The formula is:
- $Swish(x) = x \cdot \sigma(x)$
- Where: $\sigma(x) = \frac{1}{1+e^{-x}}$
- So:
- **Swish = x * sigmoid(x)**
- **Unlike ReLU, it is smooth, non-monotonic, and learns better in deep**

What is the Swish Activation Function?

- Swish is a **self-gated activation function** created by **Google Brain researchers**.
- The formula is:
- $Swish(x) = x \cdot \sigma(x)$
- Where: $\sigma(x) = \frac{1}{1+e^{-x}}$
- So:
- **Swish = x * sigmoid(x)**
- **Unlike ReLU, it is smooth, non-monotonic, and learns better in deep networks.**



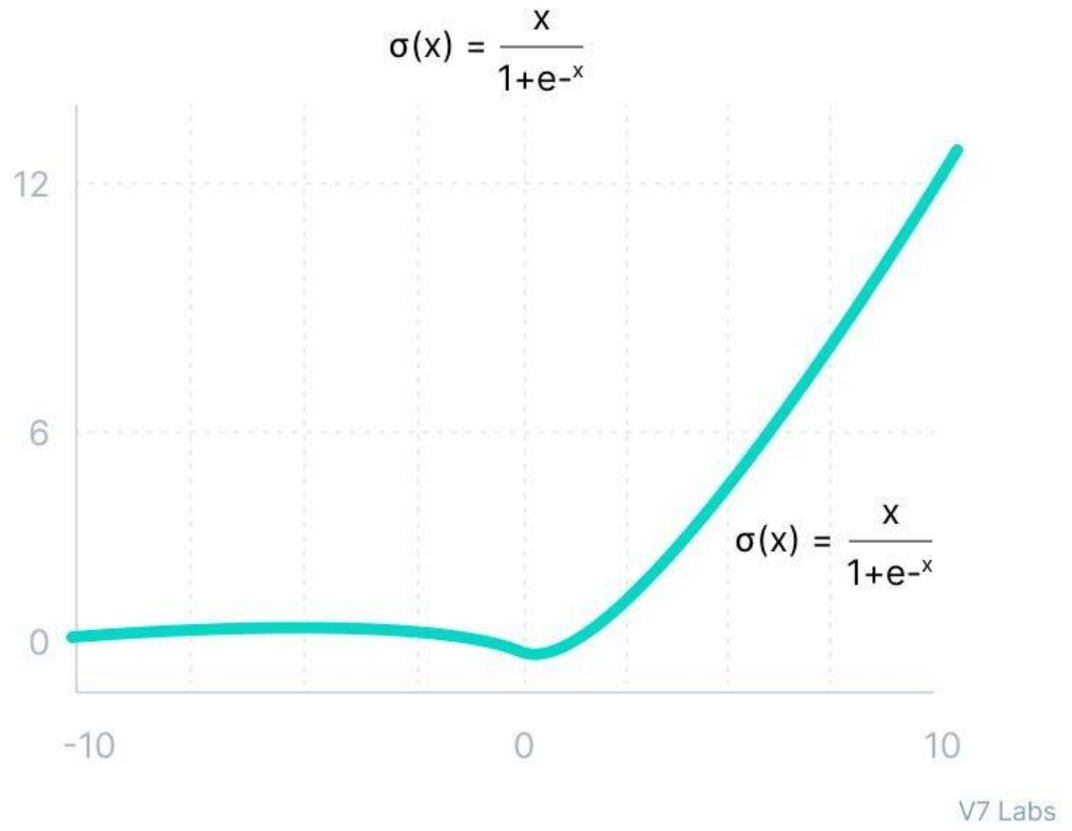
What is the Swish Activation Function?

- Swish is a **self-gated activation function** **networks.** created by **Google Brain researchers**.
- The formula is:
- $Swish(x) = x \cdot \sigma(x)$
- Where: $\sigma(x) = \frac{1}{1+e^{-x}}$
- So:
- **Swish = x * sigmoid(x)**
- **Unlike ReLU, it is smooth, non-monotonic, and learns better in deep**

What is the Swish Activation Function?

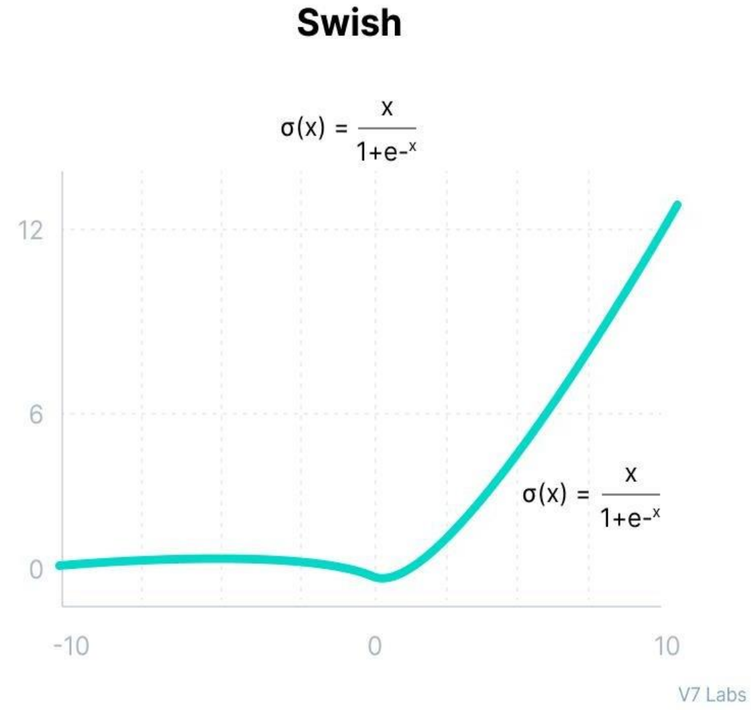
- Graph shows:
- For large **positive x**, Swish behaves almost like **ReLU**
  (output goes up without bound)
- For **negative x**, Swish does NOT go straight to 0
  (unlike ReLU)
- Instead, it forms a **smooth curve** that goes slightly **below zero**
  (this is why it's *non-monotonic*)

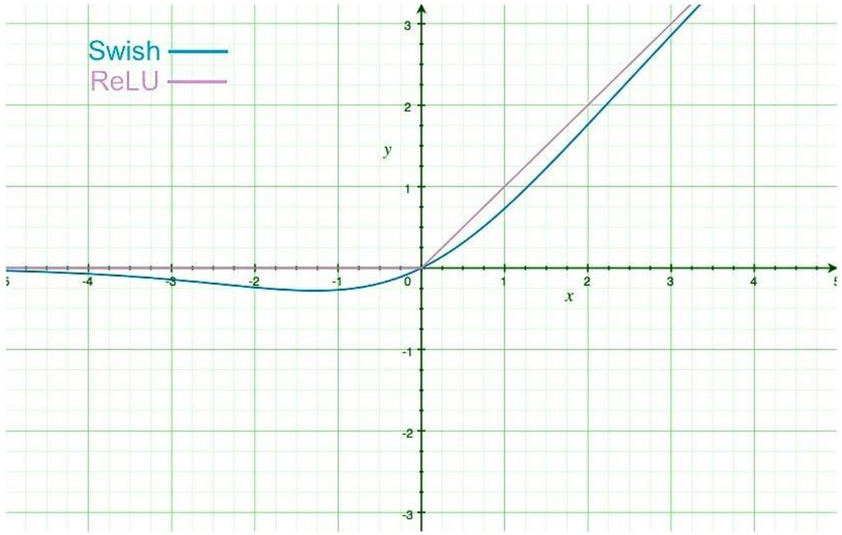
- This gentle bend is the key to Swish's power.



Swish Activation Function?

- For large **positive x**, Swish behaves almost like **ReLU**
  (output goes up without bound)
- For **negative x**, Swish does NOT go straight to 0
  (unlike ReLU)
- Instead, it forms a **smooth curve** that goes slightly **below zero**
  (this is why it's *non-monotonic*)
- This gentle bend is the key to Swish's power.





Why Swish Works Better Than ReLU

- **1. Smooth curve**
- ReLU has a harsh kink at x = 0.
  Swish is fully smooth → smoother gradient flow.

2. Non-zero negative outputs

- ReLU kills all negative values (dead neurons).
  Swish passes a bit of negative information → better learning.

3. Non-Monotonic property

- Part of Swish dips slightly below zero.
  This helps with:
- feature learning
- representation quality
- optimization stability

- **4. Self-gating**
- Swish(x) = x * sigmoid(x)
- The sigmoid depends on x, so the neuron **gates itself**.
- This is more adaptive than ReLU.

Why Swish Works Better Than ReLU

- **1. Smooth curve**
- ReLU has a harsh kink at x = 0.
  Swish is fully smooth → smoother gradient flow.

2. Non-zero negative outputs

- ReLU kills all negative values (dead neurons).
  Swish passes a bit of negative information → better learning.

3. Non-Monotonic property

- Part of Swish dips slightly below zero.
  This helps with:
- feature learning
- representation quality
- optimization stability

- **4. Self-gating**
- Swish(x) = x * sigmoid(x)
- The sigmoid depends on x, so the neuron **gates itself**.
- This is more adaptive than ReLU.

Why Sigmoid Is Not Enough for Multi-Class Output

- Sigmoid produces values between **0 and 1** for EACH neuron:
- But if we have 5 output neurons, sigmoid might produce:
- [0.8, 0.9, 0.7, 0.8, 0.6]

❌ **Problem:**

- These values do NOT sum to 1.
- So:
- 0.8 is NOT “80% probability of class 1”
- 0.9 is NOT “90% probability of class 2”
- You cannot compare these numbers as probabilities
- They violate the rule: **probabilities must sum to 1**
- So sigmoid is NOT suitable for multi-class classification.

Softmax Solves This Problem

- Softmax converts raw outputs (called logits) into **real probabilities**.
- $Softmax(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$
- This does 3 things:
- **1. Exponentials make the biggest value grow dramatically**
- (e.g., $e^3 = 20$, $e^5 = 148$)
- **2. Normalization divides by the total**
- so everything becomes a probability
- **3. All outputs sum to 1**
- so they form a valid probability distribution

Example You Gave: 3 Classes

- Output from the network: [1.8, 0.9, 0.68]
- Let's compute softmax step by step.
- **Step 1: exponentiate each:**
- $e^{1.8} \approx 6.05$
- $e^{0.9} \approx 2.46$
- $e^{0.68} \approx 1.97$
- **Step 2: sum them:**
- $6.05 + 2.46 + 1.97 = 10.48$
- **Step 3: divide each by the total:**
- 6.05 / 10.48 ≈ **0.58**
- 2.46 / 10.48 ≈ **0.23**
- 1.97 / 10.48 ≈ **0.19**
- So the probabilities are:
- [0.58, 0.23, 0.19]
- Now they sum to 1.

How Softmax Chooses a Class

- Since 0.58 is the **highest** probability:
- predicted class = class 0
- because softmax says "I am 58% confident"
- **Softmax gives:**
- 1 for the highest (winner)
- 0 for others (losers)
- ONLY during *argmax* (class selection)
  - Example: For a list [10, 30, 5, 20], the max is , while argmax returns index (position of).
- But the actual output is always probabilities like
  - [0.58, 0.23, 0.19].

Vanishing Gradients (Very Small Gradients)

- This problem happens mostly with **sigmoid**, **tanh**, and any activation that:
- squashes a large input range into a very small output range (like 0–1 or –1–1)
- Examples of squashing:
  For sigmoid:
  - x = –10 → output ≈ 0
  - x = +10 → output ≈ 1
  - gradient ≈ 0 in both cases
- **A big input change → tiny output change**
- **Tiny output change → tiny derivative**
- **Tiny derivative → tiny gradient**
- **Why is this a problem?**
- In deep networks, the gradient is multiplied many times during backprop:
  - $final\ gradient = 0.1 \times 0.1 \times 0.1 \times \cdots$
- If you multiply small numbers many times:
  - 0.1 × 0.1 × 0.1 × 0.1 = 0.0001
  - 0.01 × 0.01 × 0.01 = 0.000001
- This becomes **almost zero**.
- This means:
  - **Early layers stop learning**
  - **Network becomes slow**
  - **Training gets stuck**
  - **Can't learn long-range dependencies**
- This is why **deep networks couldn't be trained before ReLU**.

Intuition Example: Sigmoid

- Sigmoid output range is 0 to 1.
- For big inputs:
  - x = 4 → derivative ≈ 0.018
  - x = 6 → derivative ≈ 0.002
  - x = –5 → derivative ≈ 0.006
- These derivatives are **tiny**.
- So gradients "fade away" → **vanish**.
- This happens EVERY layer → huge problem.

Exploding Gradients (Very Large Gradients)

- This is the opposite of vanishing gradients.
- Instead of gradients becoming tiny, they become **huge**.
  - **Big error**
  - **Big derivative**
  - **Big gradient**
  - **Big weight update**
- Weights become too large:
  - Layer 1 → weights = 100
  - Layer 2 → weights = 5000
  - Layer 3 → weights = 90,000
  - Layer 4 → weights = 1,000,000
- This causes:
  - ❌ **Network becomes unstable**
  - ❌ **Loss becomes NaN**
  - ❌ **Weights overflow**
  - ❌ **Model crashes**

Intuition Example:

- For tanh or ReLU:
- If many layers produce outputs like:
- 5, 10, 20, 50, 100, 200, 500
- Gradients will multiply into very large numbers.
- The backward pass becomes:
- $50 \times 30 \times 20 \times 10 = 300{,}000$
- The network “explodes”.
- **How Exploding Gradients Break the Model**
  - ✓ **The weights become VERY big**
- (e.g., 1e30)
  - ✓ **Output becomes VERY big**
- (e.g., 1e20)
  - ✓ **Loss becomes VERY big**
- (e.g., 1e15)
  - ✓ **Then loss becomes NaN**
  - (not a number — network died)
- This usually happens in:
  - RNNs
  - Deep linear layers
  - Improperly initialized weights
  - High learning rates

Difference in One Sentence

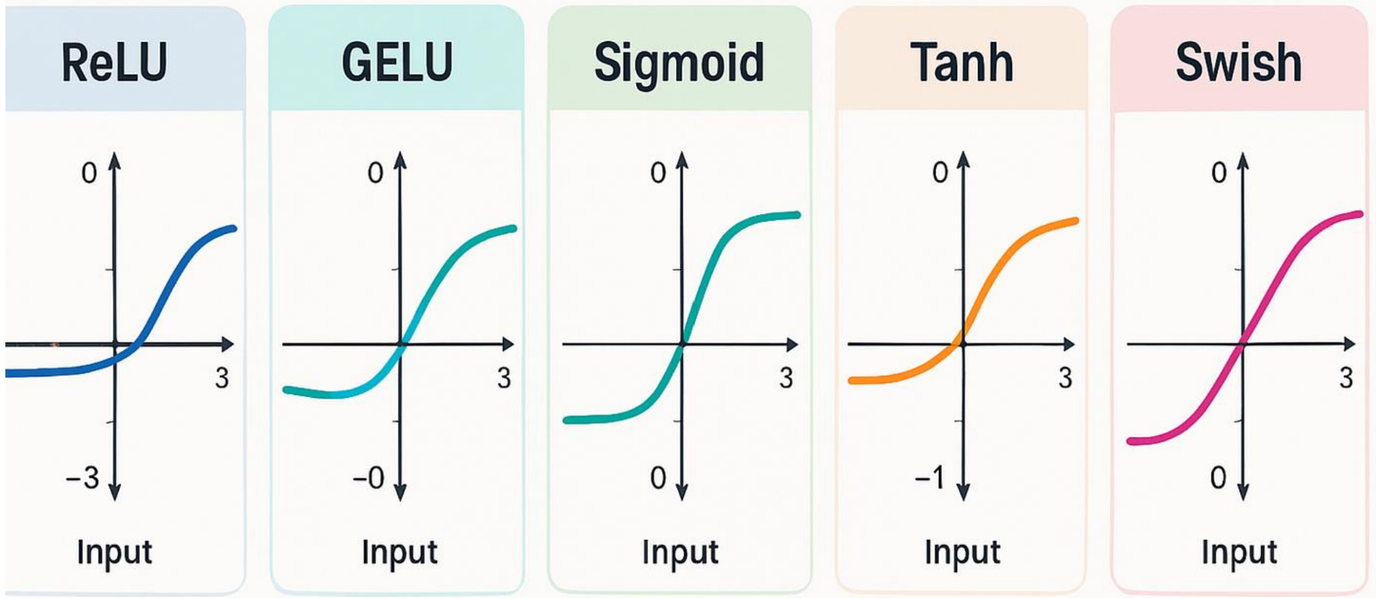
- **Vanishing Gradients = gradients shrink to near zero → no learning.**
  Exploding Gradients = gradients grow too large → unstable learning.

How to choose the right Activation Function?

- **ReLU** activation function should only be used in the hidden layers.
- **Sigmoid/Logistic and Tanh** functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
- **Swish function** is used in neural networks having a depth greater than 40 layers.
- Finally, a few rules for choosing the activation function for your output layer based on the type of prediction problem that you are solving:
- **Regression** - Linear Activation Function
- **Binary Classification** - Sigmoid/Logistic Activation Function
- **Multiclass Classification** - Softmax
- The activation function used in hidden layers is typically chosen based on the type of neural network architecture.
- **Convolutional Neural Network (CNN)**: ReLU activation function.
- **Recurrent Neural Network**: Tanh and/or Sigmoid activation function.

How to choose the right Activation Function?

ReLU $\max(0, x)$	**GELU** $\frac{x}{2}\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}\left(x+ax^3\right)\right)\right)$	**PReLU** $\max(0, x)$
ELU $\begin{cases} x \text{ if } x > 0 \\ \alpha(x \exp x - 1) \text{ if } x < 0 \end{cases}$	**Swish** $\frac{x}{1+\exp -x}$	**SELU** $\alpha(\max(0, x)+ \min(0, \beta(\exp x - 1)))$
SoftPlus $\frac{1}{\beta}\log\left(1+\exp(\beta x)\right)$	**Mish** $x\tanh\left(\frac{1}{\beta}\log\left(1+\exp(\beta x)\right)\right)$	**RReLU** $\begin{cases} x \text{ if } x \geq 0 \\ ax \text{ if } x < 0 \text{ with } a \sim \Re(l, u) \end{cases}$
HardSwish $\begin{cases} 0 \text{ if } x \leq -3 \\ x \text{ if } x \geq 3 \\ x(x+3)/6 \text{ otherwise} \end{cases}$	**Sigmoid** $\frac{1}{1+\exp(-x)}$	**SoftSign** $\frac{x}{1+\lvert x\rvert}$
Tanh $\tanh(x)$	**Hard tanh** $\begin{cases} a \text{ if } x \geq a \\ b \text{ if } x \leq b \\ x \text{ otherwise} \end{cases}$	**Hard Sigmoid** $\begin{cases} 0 \text{ if } x \leq -3 \\ 1 \text{ if } x \geq 3 \\ x/6+1/2 \text{ otherwise} \end{cases}$
Tanh Shrink $x - \tanh(x)$	**Soft Shrink** $\begin{cases} x-\lambda \text{ if } x > \lambda \\ x+\lambda \text{ if } x < -\lambda \\ 0 \text{ otherwise} \end{cases}$	**Hard Shrink** $\begin{cases} x \text{ if } x > \lambda \\ x \text{ if } x < -\lambda \\ 0 \text{ otherwise} \end{cases}$



How to choose the right Activation Function?

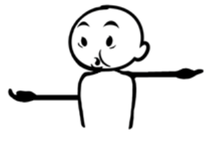
Sigmoid

$$y = \frac{1}{1+e^{-x}}$$

Tanh

$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus

$$y = \ln(1+e^{x})$$

ReLU

$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign

$$y = \frac{x}{(1+|x|)}$$

ELU

$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid

$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish

$$y = \frac{x}{1+e^{-x}}$$

Sinc

$$y = \frac{\sin(x)}{x}$$

Leaky ReLU

$$y = \max(0.1x, x)$$

Mish

$$y = x(\tanh(\text{softplus}(x)))$$

How to choose the right Activation Function?

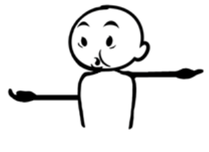
Sigmoid

$$y = \frac{1}{1+e^{-x}}$$

Tanh

$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus

$$y = \ln(1+e^{x})$$

ReLU

$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign

$$y = \frac{x}{(1+|x|)}$$

ELU

$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid

$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish

$$y = \frac{x}{1+e^{-x}}$$

Sinc

$$y = \frac{\sin(x)}{x}$$

Leaky ReLU

$$y = \max(0.1x, x)$$

Mish

$$y = x(\tanh(\text{softplus}(x)))$$

What Is a Loss Function?

- **Loss function** measures:
- **How wrong the model's prediction is for ONE training example.**
- Example:
- You have a neural network that predicts car prices.
  - True price = **$35,000**
  - Predicted price = **$40,000**
  - Loss = difference between true & predicted.

    Depending on the loss function type:
- **MAE:** |40,000 – 35,000| = 5,000
- **MSE:** $(40,000 – 35,000)^2$ = 25,000,000
- This number tells us **how bad the prediction is**.
- **Loss = error for one training example**
- **Used by backpropagation to update weights**

How Loss Is Used in Training

- Backpropagation computes:
  - $gradients = \frac{\partial loss}{\partial\, weights}$
- This tells the network:
  - how to update weights
  - how to reduce error
  - how to improve performance
- So:
  - **Loss → gradients → weight updates → learning**
- This is why loss is essential.

What Is a Cost Function?

- A **cost function** is simply:
- **The average loss over the entire training dataset.**
- If you have 600 training examples:
- $Cost = \frac{1}{600}\sum_{i=1}^{600} \text{Loss}_i$
- This gives one single number that summarizes:
- how well the model performs on **all training data**
- the goal of training = **minimize the cost**
- **Why Both Are Needed?**

- **Loss function**
  - Needed for **backpropagation**.
  - Gradient is computed from the loss of each sample (or batch).
- **Cost function**
  - Needed to measure **overall model performance**.
  - Used during training to:
  - track progress
  - adjust learning rate
  - decide when to stop

MEAN SQUARED ERROR (MSE or L2 Loss)

Measures the average of squared differences

- MSE $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- Where:
  - y_i= actual value
  - $\hat{y}_i$= predicted value
  - n= number of samples

Why do we square the error? (Intuition)

- Because squaring makes **big errors much bigger**.
- Example:
  - Error 5 becomes 25
  - Error 10 becomes 100
- So:
- **MSE gives heavy penalty to large mistakes**
- **Good when you want the model to be extra careful about big errors**

Smooth and differentiable → easier for gradient descent

- This is why MSE is the most common loss in regression.

When MSE is Good

- Use MSE when:
  - You want to punish big errors
  - Data has no extreme outliers
  - You need smooth gradients for faster learning
  - Your prediction needs to be precise
- Examples:
  - Predicting house prices
  - Forecasting continuous values
  - High-accuracy regression tasks

Mean Absolute Error (MAE or L1 Loss)

- **Measures the average of absolute differences**
- MAE $= \frac{1}{n}\sum_{i=1}^{n} | y_i - \hat{y}_i |$
- **Why absolute difference? (Intuition)**
- Because MAE treats **all errors equally**.
- Error 5 → 5
  Error 10 → 10
- MAE does **not** exaggerate large errors.
- So:
- **MAE is more robust to outliers**
- **MAE is more stable if your data has spikes or noise**

When MAE is Good

- Use MAE when:
- Your data has outliers (noise)
- You don't want big mistakes to dominate the loss
- You want a more "balanced" error measure
- Examples:
- Predicting traffic speed (lots of noise)
- Medical data with measurement errors
- Financial data with spikes

What is Huber Loss?

- **Huber Loss** is a special loss function used in regression that:
- **behaves like MSE (Mean Squared Error) when the error is small**
- **behaves like MAE (Mean Absolute Error) when the error is large**
- So it **combines the best parts** of MSE and MAE.
- This is why Huber Loss is called a **robust** loss function.
- **Why Combine MSE and MAE?**
- Because:
- **MSE**
- Penalizes large errors **too strongly**
- Very sensitive to outliers
- Can explode
- **MAE**
- Handles outliers well
- But not smooth at 0
- Gradient is constant → slow learning
- So Huber Loss mixes them:
- **Small error → behave like MSE (fast learning)**
- **Large error → behave like MAE (resistant to outliers)**
- Best of both worlds.

Huber Loss

- Let the error (residual) be:
- $a = y - f(x)$
- Huber loss:
- $L_\delta(a) = \begin{cases} \frac{1}{2}a^2, & \text{if } |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta), & \text{if } |a| > \delta \end{cases}$
- Where:
- a= difference between actual and predicted value
- δ= threshold (a hyperparameter)
- **If the error is small (within δ)**
- Use **squared error** (like MSE): $\frac{1}{2}a^2$
- This keeps training **smooth** and **stable**.
- **If the error is large (greater than δ)**
- Use **absolute error** (like MAE):,$\delta(|a| - \frac{1}{2}\delta)$
- This prevents **outliers** from blowing up the loss.

Why Huber Loss is Better Than MSE and MAE

- **More robust to outliers than MSE**
  - MSE squares errors → outliers dominate
    Huber → switches to MAE when errors get big
- **Smoother than MAE**
  - MAE is not differentiable at 0
    Huber is completely smooth → better gradients
- **Faster training**
  - Because for small errors, it behaves like MSE
    → gradients are stronger
    → optimization is faster

Huber Loss

- **Example to Understand Huber**
- Let δ = 1.
- **Case 1: Small error**
- Prediction = 10
  Actual = 10.5
  Error = –0.5
- $|a| \leq 1$ ⇒use MSE
- $L = \frac{1}{2}(0.5)^2 = 0.125$
- **Case 2: Large error**
- Prediction = 10
  Actual = 15
  Error = 5
- $|a| > 1$ ⇒use MAE-like form
- $L = 1(5 - 0.5) = 4.5$
- Instead of squaring the error (25), Huber reduces the punishment (4.5).

Where Huber Loss Is Used

  - **Regression with noise**
  - **Datasets with outliers**
  - **Robust machine learning systems**
  - **Time-series forecasting**
  - **Autonomous vehicle sensor data**
  - **Training deep learning models that need stability**

- Many real-world models use Huber Loss because of its balance.

Binary Cross-Entropy (BCE) Loss?

- Binary Cross-Entropy (also called **Log Loss**) is the **best loss function for binary classification**, where the target can only be:
- **0** (negative class), or
- **1** (positive class)
- It measures **how different** the predicted probability is from the **true label**.
- **Why BCE Is Needed (Intuition)**
- A model predicts a **probability**:
  - If f(x) = 0.90 → 90% chance the sample belongs to Class 1
  - If f(x) = 0.10 → 10% chance the sample belongs to Class 1
- We want the model to:
  - Give probabilities close to **1** when the true label is **1**
  - Give probabilities close to **0** when the true label is **0**
  - Binary Cross-Entropy measures **how bad** the prediction is.
- **BCE Formula**
  - $L\big(y, f(x)\big) = -\big[y \cdot \log\big(f(x)\big) + (1-y) \cdot \log\big(1 - f(x)\big)\big]$
- Where:
- **y** = actual label (0 or 1)
- **f(x)** = predicted probability (between 0 and 1)

Understanding the Formula

Case 1: When the true label is 1

- (y = 1)
  - $L = -\log(f(x))$
- If the model predicts:
  - f(x) = 0.9 → Loss = LOW (good model)
  - f(x) = 0.01 → Loss = HUGE (terrible model)
- So BCE punishes wrong predictions heavily.
- **Case 2: When the true label is 0**
- (y = 0)
  - $L = -\log(1 - f(x))$
- If the model predicts:
- f(x) = 0.1 → Loss = LOW (good)
- f(x) = 0.99 → Loss = HUGE (very wrong)
- BCE encourages predicted probability to match the label.

Why BCE uses log()?

- Because **log** grows rapidly when the model is confident but wrong.
- Example:
- log(0.99) ≈ –0.01 → very small
- log(0.01) ≈ –4.6 → very large penalty
- This forces the model to **not make confident mistakes**.
- **Why BCE is perfect for binary classification**
  - **Outputs probability between 0 and 1**
  - **Strong punishment for confident wrong predictions**
  - **Smooth gradients for backpropagation**
  - **Works perfectly with sigmoid activation in the last layer**
  - **Stable, mathematically sound loss function**

What Is Categorical Cross-Entropy?

- Categorical Cross-Entropy (CCE) is the **standard loss function** for **multi-class classification**, where you have:
- More than 2 classes
- Only **one** correct class out of many
- Output layer uses **Softmax**
- Examples:
- Classifying 10 digits (0–9)
- Classifying animals (cat, dog, horse, elephant)
- Classifying documents into topics

Why Softmax Must Be Used Before CCE?

- Softmax converts logits into **probabilities** that:
- lie between 0 and 1
- and **add up to 1**
- Example Softmax output:
- [0.58, 0.23, 0.19]
- Each value is the probability of a class.
- Categorical Cross-Entropy compares:
  - **the predicted probabilities**
  - **the true one-hot vector**

What Is Categorical Cross-Entropy?

- Categorical Cross-Entropy (CCE) is the **standard loss function** for **multi-class classification**, where you have:
- More than 2 classes
- Only **one** correct class out of many
- Output layer uses **Softmax**
- Examples:
- Classifying 10 digits (0–9)
- Classifying animals (cat, dog, horse, elephant)
- Classifying documents into topics

Why Softmax Must Be Used Before CCE?

- Softmax converts logits into **probabilities** that:
- lie between 0 and 1
- and **add up to 1**
- Example Softmax output:
- [0.58, 0.23, 0.19]
- Each value is the probability of a class.
- Categorical Cross-Entropy compares:
  - **the predicted probabilities**
  - **the true one-hot vector**

What Is Categorical Cross-Entropy?

- What Is a One Hot Vector?
- If you have 3 classes:
- Class: One-hot label
  - **Class 0** [1, 0, 0]
  - Class 1[0, 1, 0]
  - Class 2[0, 0, 1]
- Example:
- If the true class = 0
- the true label is: y=[1,0,0]

Categorical Cross-Entropy

- If you have **k classes**:
- $Loss = -\sum_{i=1}^{k} y_i \cdot \log(p_i)$

  Where:
  - y_i= 1 only for the correct class
  - p_i= predicted probability for class i

- Because only one class has $y_i = 1$:
  CCE simply becomes:
- $Loss = -\log(predicted\ probability\ of\ correct\ class)$

  Predicted Softmax probabilities:
  - $[0.58, 0.23, 0.19]$
- True class = 0 → one-hot = [1,0,0]
- **CCE:**
  - $L = -\log(0.58)$
- If the model gave good probability → loss is small.
- If the model predicted badly (e.g. 0.01 for correct class) → $\log(0.01)$ =large negative → loss becomes **huge**.

What Is Hinge Loss?

- **Hinge Loss** is a loss function used mainly in **maximum-margin classification**, especially in **Support Vector Machines (SVMs)**.
- It encourages the model to:
  - **classify correctly**
  - **AND with a large margin (high confidence)**
- Unlike cross-entropy (which focuses on probabilities),
  Hinge Loss focuses on raw scores and margins.
- **Why "Margin" Is Important?**
- A margin is the **distance between the data point and the decision boundary**.
- We want:
  - Correctly classified points → **far** from the boundary
  - Incorrectly classified points → **penalized strongly**
- Correct but close points → **still penalized** (encourages larger margins)
- This improves **generalization** → the model performs better on unseen data.

Hinge Loss Formula

- $L(y, f(x)) = \max(0,\ 1 - y \cdot f(x))$
- Where:
- **y** ∈ {–1, +1} (true class label)
- **f(x)** = model's raw output score (before sigmoid/softmax)
- **Note:** Hinge loss uses labels –1 and +1 (not 0 and 1 like BCE).

Case 1: Correct with high confidence

- Suppose y = +1 and f(x) = +3
- $1 - yf(x) = 1 - 3 = -2$
- $L = \max(0, -2) = 0$
  - Loss = **0**
  - No penalty
  - Model predicted strongly and correctly

Case 2: Correct but close to boundary (small margin)

- y = +1, f(x) = 0.4
- $1 - yf(x) = 1 - 0.4 = 0.6$
- $L = 0.6$
- Model is correct, BUT too close to decision boundary
  Loss > 0 → model is penalized
  Pushes model to create larger margin
- **Case 3: Wrong classification**
- y = +1, f(x) = –2
- $1 - yf(x) = 1 - (-2) = 3$
- $L = 3$

  Loss = 3
- High penalty for being confidently wrong